Revision — 1.2.4 Types of Programming Language

OCR H446 — one-page revision summary

Must know

- A **programming paradigm** is a **style or approach** to writing programs. No single language suits every task, so pick the paradigm that makes a problem easier to write, maintain and reuse.
- **Procedural** programming uses ordered statements built from **sequence, selection and iteration**, organised into **procedures/subroutines**; it specifies *how* to solve the problem (Python, C, Pascal).
- **Assembly language** is **low-level** and uses **mnemonics** (e.g. `ADD`, `STA`), with roughly **one mnemonic per machine instruction**. It is **processor-specific** and chosen for speed, small size or direct hardware control (device drivers, embedded systems).
- The **Little Man Computer (LMC)** is a simple model CPU and instruction set used to learn assembly and the fetch–decode–execute cycle. Key instructions: `INP`, `OUT`, `STA`, `LDA`, `ADD`, `SUB`, `BRA` (jump always), `BRZ` (branch if ACC = 0), `BRP` (branch if ACC ≥ 0), `DAT`, `HLT`.
- **Addressing modes** determine how the operand is interpreted: **Immediate** (operand *is* the value), **Direct** (operand is the address holding the value), **Indirect** (operand is the address of a location that holds the **address** of the value — pointers), **Indexed** (effective address = base address + **index register** contents — used for arrays).
- **Object-oriented programming (OOP)** models a problem as **objects**. A **class** is a template/blueprint of attributes + methods; an **object** is a specific **instance** of a class; **instantiation** is creating an object from a class.
- An **attribute/property** is a **variable** of a class (its data/state); a **method** is a **subroutine** of a class (its behaviour).
- **Encapsulation** bundles data and its methods together and **hides the data** (private attributes accessed only through methods) — protecting data and easing maintenance.
- **Inheritance** lets a **subclass** acquire the attributes and methods of a **superclass** (and add or override them) — reducing duplicated code (DRY). **Polymorphism** lets the same method name behave differently depending on the object (typically via overriding).
- **Class diagram:** a box split into class name / attributes / methods; `+` = public, `-` = private; the inheritance arrow points from **child → parent**.

Key terms

Term	Definition
Programming paradigm	A style or approach to writing programs
Assembly language	Low-level, processor-specific language using mnemonics
Class	A template/blueprint defining attributes and methods
Object	A specific instance of a class
Attribute	A variable of a class/object (its data/state)
Method	A subroutine of a class (its behaviour)
Encapsulation	Bundling data with its methods and hiding the data (private attributes)
Inheritance	A subclass acquiring the attributes/methods of a superclass

Worked example / how to — addressing modes

Instruction `LDA 6`, where **address 6 holds 9**, **address 9 holds 4**, and the **index register holds 2**: - **Immediate → 6** — the operand is the value itself. - **Direct → 9** — load the value stored at address 6. - **Indirect → 4** — address 6 holds 9, so go to address 9, whose value is 4. - **Indexed → effective address = 6 + 2 = 8** — base address 6 plus index 2; the value is taken from address 8.

Exam tips

- Say why a paradigm suits a task — e.g. **assembly** for speed/size/hardware control; **OOP** for reuse via inheritance and polymorphism.
- For addressing modes, **show the working**: immediate = value, direct = one lookup, indirect = two lookups (pointer), indexed = base + index register.
- Keep OOP definitions crisp: **class = blueprint, object = instance, attribute = variable, method = subroutine**; encapsulation includes **data hiding**.
- Polymorphism = same method name, different behaviour per object (e.g. `display()` differs for Car vs Bus); link it to inheritance and the DRY benefit.